

State & storage: where conversation state lives

How to read this file: the first half is GENERAL state/storage knowledge (transferable to any system/job/interview). The second half (marked "WORKED EXAMPLE") is how the WCX repo instantiates it. Learn the general model; use the repo to make it concrete. **Trust status:** `build_session_doc` fields + the thread/Cosmos split verified against code across Days 4-7. The SRE **thread** is a managed-service store (its schema is Azure's, not this repo's) — this repo only holds the `thread_id` pointer. The 3-IDs model and reload path are code-backed. General concepts are stable industry knowledge. **Why this file exists:** the #1 source of confusion was "where does history actually live" — this file pins down the distinct stores (managed thread / Cosmos metadata / in-memory client) so "does X survive reload?" has a definite answer. **Related:** `[[kb-agent-loop]]` (thread = the LLM's context; the loop's state lives in the store), `[[kb-streaming-pipeline]]` (the thread is what gets polled), `[[kb-frontend-architecture]]` (client-side transcript), `[[kb-distributed-systems-patterns]]` (statelessness, idempotency, at-least/at-most-once).

PART 0 — WHY THIS MATTERS (philosophy & real-world connection)

- **Deep principle in one line:** keep compute stateless and push state to one durable store — so any worker can serve any request, and a crash loses nothing.
 - **Why it exists / what it solves:** this is the trick that makes web backends scale horizontally and survive failure. If no request depends on which box handled the *last* request, the load balancer is free to send traffic anywhere and you scale by cloning identical instances. Crucially, "stateless" doesn't mean "no state" — it means no state *in the process*. The state still exists; it just lives in a shared durable store, not in one server's RAM. The whole thing rests on one pattern: **client = disposable view, server = durable record, an id links them**. That triangle is the backbone of essentially every web app I'll ever touch.
 - **Real-world connection beyond this repo:** I meet this everywhere once I look for it. Every SaaS app. ChatGPT's `/c/<id>` URL — the id in the path is the seam that lets a reload re-fetch my history. REST APIs sending self-contained requests. Redis session stores holding server-side sessions. The SQL-vs-NoSQL paragraph in every design doc. CDN and cache layers accelerating reads without becoming the source of truth. Why auth tokens live in cookies (auto-attached to every request) and not localStorage. The metadata-vs-content split shows up as S3-blob-plus-a-DB-row, and as video platforms keeping a metadata DB separate from the blob store. This is bread-and-butter system-design-interview material — if I can reason about it fluently, a huge class of design questions gets easier.
 - **What breaks without it:** sticky sessions that pin a user to one box and won't scale; losing a whole conversation the instant that instance dies; jamming the huge transcript into the frequently-listed index so every sidebar load drags the payload and goes slow; confusing cookie vs localStorage and either leaking an auth token or losing state I thought I saved; and no id in the URL, so after a reload there's nothing to re-fetch *from* and the chat is just gone.
 - **Transferable mental model:** state lives SOMEWHERE durable, keyed by an id; compute is a stateless function over (request + fetched state). Whenever I look at any feature, I should be able to answer crisply: **"Where does X live, and does it survive a reload?"** If that answer is fuzzy, the design has a hole.
-

PART 1 — GENERAL KNOWLEDGE

1.1 The stateless-server principle (and why it dominates)

A **stateless server** keeps NO per-client memory between requests: every request carries (or references) everything needed to handle it, so **any server instance can handle any request**. This is the default target for web backends because of what it buys you:

- **Horizontal scalability:** add more identical instances behind a load balancer; no instance is "special" for a given user.
- **Load-balancing freedom:** the balancer can route each request anywhere (round-robin, least-conn) — no sticky sessions required.
- **Crash-resilience:** if an instance dies mid-conversation, the next request lands on a healthy one and continues, because the *state was never in that instance's RAM* — it was in a shared store.

The cost: the state has to live *somewhere durable and shared*, and every request pays to re-load it. Stateless doesn't mean "no state" — it means "no state **in the server process**." The state is externalized. See `[[kb-distributed-systems-patterns]]`.

1.2 Where state can live (the whole map)

Rank these by durability and by who can see them:

Location	Durability	Scope / who sees it	Typical use
Client memory (JS vars, React state)	dies on reload/navigate	one tab	live UI scratchpad, optimistic view
URL (path/query/hash)	survives reload; shareable	anyone with the link	which resource is open (/c/<id>), filters
Cookies	survives reload; sent every request	client + server (auto-attached)	auth/session token, small prefs
localStorage / sessionStorage	localStorage persists; session dies on tab close	one origin, client-only (not auto-sent)	draft text, client cache, feature flags
Server session store (Redis, etc.)	as durable as the store	server-side, keyed by session id	logged-in user's server-side session
Database (SQL / NoSQL)	durable, backed up	server-side, the source of truth	the actual records (messages, users)

The **cookie vs localStorage** distinction trips people up: a cookie is **automatically attached to every HTTP request** to that domain (which is why auth tokens live there), whereas localStorage is client-only and you must manually put its contents into a request. Cookies are for "the server needs this every time"; localStorage is for "the client wants to remember this."

1.3 The universal pattern: client = disposable view, server = durable record, an id links them

The single most important mental model for stateful apps:

Client state is a live, disposable VIEW; the server holds the durable RECORD; an ID (usually in the URL) is the pointer that reconnects them.

Concretely with ChatGPT: the URL is `/c/<conversation-id>`. The React app in your tab is disposable — reload and it resets to empty. But the `<conversation-id>` in the URL survives the reload, and on load the app fetches that conversation from the server, so your history reappears. The **id is the seam** between the ephemeral client and the durable server. This pattern is everywhere: a document editor (`/doc/<id>`), an email thread, an issue tracker. Lose the client → you lose nothing; lose the id → you can't find your record.

Persistence across page reload is exactly this pattern in action: reload nukes client memory, so anything that must survive has to be either (a) reconstructable from the URL id by re-fetching, or (b) already written to a client-durable store (localStorage) or server store.

1.4 SQL vs NoSQL / document stores (and when to use which)

- **SQL / relational** (Postgres, MySQL, SQL Server): fixed schema, tables + rows, strong joins, ACID transactions, great when data is highly relational and you query across entities. Cost: schema migrations, harder to scale writes horizontally.
- **NoSQL document stores** (Cosmos DB, DynamoDB, MongoDB): store JSON-ish **documents**; schema-flexible, partitioned by a key for near-infinite horizontal scale, cheap fast lookups by key. Weak/absent joins; you often **denormalize** (duplicate data) instead. Great for "give me this user's list of X by their id."
 - **Cosmos DB** = Azure's globally-distributed multi-model store; here used as a document (NoSQL) store. A "**document**" (`doc`) = one JSON record.
 - **DynamoDB** = AWS's key-value/document store; **MongoDB** = the popular open-source document DB.
- **Rule of thumb**: relational + transactional + lots of cross-entity queries → SQL. Massive scale, key-based access, flexible/evolving shape, "index of things owned by X" → document store. Many real systems use both.

1.5 Metadata store vs content store (and why you'd split them)

A recurring architecture: put **metadata** in one store and **content** in another.

- **Metadata store** = small, structured facts *about* a thing: title, owner, timestamps, tags, a **pointer** to the content. Read constantly (list views, sidebars), changes slowly.
- **Content store** = the big/heavy payload itself: the full document, the transcript, the blob. Read on demand, may change rapidly during a session.

Why split (**separation of concerns**):

- **Different access patterns**: you list metadata far more often than you open content; you can index/query metadata cheaply without dragging the payload.
- **Different change rates**: a title changes rarely; a live transcript grows every second. Keeping the fast-changing bulk out of the frequently-listed index keeps both efficient.
- **Different stores fit different jobs**: metadata → a queryable index (SQL or a document store keyed by owner); content → a big append-friendly store or a managed service.

1.6 Pointers, foreign keys, and references across stores

When state is split across stores, you connect them with a **pointer** — the same idea as a **foreign key** in SQL, generalized. The metadata record holds the **id** of the content record; to get the content you follow the pointer with a second lookup. Across two *different* systems (e.g., a document store pointing at a managed thread service) there's no DB-enforced integrity — the pointer is just an id string you agree to honor, so orphan/dangling pointers are a real failure mode to design around. A pointer is a *reference*, not a copy: cheap to store, indirection to resolve.

1.7 Caching layers

A **cache** is a fast, usually in-memory copy of data whose source of truth lives elsewhere, kept close to the consumer to avoid a slow/expensive re-fetch. Layers stack: browser cache → CDN → app-level (Redis/memcached) → DB buffer pool. The two hard parts are **invalidation** (when does the copy go stale?) and **consistency** (a cache can serve older data than the source). Rule: a cache must never be the *source of truth* — it's a disposable accelerator, like the client view in §1.3.

1.8 Idempotency keys

An operation is **idempotent** if doing it twice has the same effect as once. When a write's HTTP response can be **lost** (network blip) even though the write *landed*, a blind retry duplicates it. An **idempotency key** — a client-generated unique id attached to the request — lets the server recognize "I already applied this key" and no-op the retry. A weaker variant is **verify-then-retry**: before retrying, check whether your write actually appeared, and only retry if it genuinely didn't. Both give **at-most-once** effect over an **at-least-once** transport. See [\[\[kb-distributed-systems-patterns\]\]](#) and [\[\[kb-streaming-pipeline\]\]](#) §1.8.

1.9 Conversation memory for a STATELESS LLM (the crux)

An LLM has **no memory between calls** — each API call is a pure function of the input you send. So "the model remembers our conversation" is an illusion: **the entire transcript must be re-sent on every call**. This raises one question — **who holds the transcript between turns?** — and there are only a few answers:

- **The client holds it** and re-sends the whole array each turn (raw OpenAI/Anthropic usage: you pass `messages=[...]`).
- **A managed service holds it** behind an id, and you send only the new turn + the id; the service reassembles the transcript server-side.

Either way, something durable accumulates the ordered messages, because the model itself won't. This is *why* stateful chat apps need a server-side conversation store at all (§1.1 + §1.3): the LLM's statelessness forces state into the storage layer. See [\[\[kb-agent-loop\]\]](#) (the loop decides *how much* of the stored transcript to send — the **context-window limit** bites at the model, not the store).

Session vs conversation vs thread — same concept, provider-dependent naming for "the server-side store of an ongoing exchange":

Provider / context	Name
Azure SRE Agent	thread (<code>/api/v1/threads</code>)
Azure AI Foundry	conversation (<code>responses.create(conversation=id)</code>)
OpenAI Assistants	thread

Raw OpenAI/Anthropic	messages array (you hold it)
Many web apps	session

Don't let the noun fool you — it's the same server-side-durable-transcript-behind-an-id idea every time. ("Session" is also overloaded: sometimes it means the auth/browser session, sometimes the conversation. Disambiguate by context.)

PART 2 — WORKED EXAMPLE: the WCX thread / Cosmos / client model

WCX is a textbook instance of §1.3 (disposable client view + durable server record + an id) and §1.5 (metadata store split from content store), driven by §1.9 (a stateless LLM forces the transcript into storage). Three distinct stores, connected by pointers.

The universal chat-app pattern, WCX-instantiated

Client state = live disposable view; server = durable record; an id (in URL) links them. Same as ChatGPT (`/c/<id>` in URL → fetch conversation on load → history reappears, §1.3). The *need* for server-side conversation state is universal (because the LLM is stateless, §1.9); only the NAME varies (§1.9 table). How I know "thread" is SRE-specific: it's a hard-coded endpoint path in *their* API (`sre_client.py`). The concept survived a real backend swap (Foundry "**conversation**" → SRE "**thread**") — the pointer stayed (`conversationId == thread_id`), the noun changed. **Thread is SRE-specific naming; the concept is universal.**

WHO holds context server-side = the THREAD (not the loop, not the LLM)

Thread = durable Azure-managed storage (a row/doc in the SRE Agent service's own DB), holds ordered messages = the conversation memory the LLM reasons over (§1.9). It's a **managed content store** (§1.5) — its schema is Azure's, not this repo's. Reached ONLY via REST (`create_thread` / `post_message` / `get_messages`); the portal holds just `thread_id` (a **pointer**, §1.6). Adding a new question to context = the SERVER's job when it gets `post_message(thread_id, text)` — the host stays **stateless** about content (§1.1). The thread is also where the agent-loop's "state" actually lives (the loop's control logic is memoryless; see `[[kb-agent-loop]]`).

COSMOS vs THREAD — both stores, different data, NOT duplicates (§1.5 made concrete)

- **Thread (Azure/SRE)** = conversation CONTENT (the messages) = the **content store**.
- **Cosmos (portal)** = session METADATA + POINTER = the **metadata store**. `build_session_doc` → `{id: "sess_...", type, userId, conversationId (=thread id), product, agentName, title, firstMessage, pinned, createdAt, updatedAt}`. **No messages array** — the transcript is NOT in Cosmos.
- Cosmos's most important job = the user's SESSION DIRECTORY/index (sidebar list, titles, recovery-after-refresh). Maps `user→sessions→thread_id`. `firstMessage` stored only for title/preview, not as the record.
- This is exactly §1.5's split: **metadata** (title/pinned/timestamps, listed constantly, changes slowly) lives in Cosmos; **content** (the growing transcript) lives in the thread. The `conversationId` field in the Cosmos doc IS the cross-store **pointer** (§1.6) from metadata → content. `doc` = "document" = a Cosmos record (JSON object); Cosmos = NoSQL document store (§1.4).

The 3 IDs (Day 4)

`user_oid` (who, from flask.g/Entra) → owns many `sessionId` (durable chat container in Cosmos: title/pinned/sidebar) → points to `conversationId` (the live LLM history in the thread — the "coat-check ticket"). One session can have MANY conversations over time (new agent version → reopen session → old messages from Cosmos but fresh thread memory). Split because state changing at different rates lives in different stores (§1.5 — different change rates).

- **Coat-check ticket:** browser sends `conversationId` → continued chat, portal passes the ticket, server holds the coat (history). No `id` → new EMPTY conversation created. (This is the §1.3 id-links-them pattern: the ticket is the pointer; lose it and you can't find your coat.)

History across turns & reload (7/16) — persistence-across-reload (§1.3) in the flesh

- **Within a session (client):** the durable client transcript = reducer `state.messages` (see `[[kb-frontend-architecture]]`), NOT `ctx.allMessages` (per-turn, discarded). A prior turn's reply reaches a later turn THROUGH `state.messages`. This is the disposable **client view** (§1.2 client memory).
- **Across reload/days:** reducer state does NOT survive (in-memory, resets to `[]` — §1.2 client memory dies on reload). Durable history = SERVER (thread content + Cosmos pointer). On reopen: sidebar lists sessions from Cosmos → click → backend

`get_messages` from thread via `conversationId` → `dispatch(Load_Session)` refills `state.messages` → all turns re-render. The conversation-id-in-URL survives refresh and lets reload find the chat (§1.3 — id is the seam). Follow the pointer (Cosmos `conversationId` → thread) to rehydrate content, §1.6.

Context-window limit is AGENT-side (Day 5)

Host has NO context concern (just streams deltas — see [\[\[kb-streaming-pipeline\]\]](#)). Thread = unlimited storage (no token limit — the store doesn't care). LLM = where the token limit bites (§1.9 — the model is stateless and bounded). The agent LOOP bridges the gap (decides how much thread history to send; truncate/summarize — see [\[\[kb-agent-loop\]\]](#)). Portal entirely out of context-management.

gevent background write — off the critical path (Day 5)

`gevent` = green-threads concurrency; `gevent.spawn(fn, arg)` = run concurrently, don't block. Used to persist the session doc (the Cosmos metadata write) **OFF the streaming critical path** so the Cosmos write doesn't delay the first token to the user; inline under debug where the gevent hub isn't driven. General lesson: metadata writes (§1.5) are cheap and non-blocking to the user's read path — push them off the latency-critical path.

PART 3 — transferable takeaways (the interview/next-job version)

1. **Stateless server** = no state in the process; externalize it so any instance can serve any request (scale, load-balance, survive crashes). "Stateless" ≠ "no state."
2. **Client view is disposable, server record is durable, an id (in the URL) links them.** Lose the client → lose nothing; lose the id → lose the record.
3. **Persistence across reload** = re-fetch from the URL id, or use a client-durable store (localStorage/cookies). Reload always nukes plain client memory.
4. **Split metadata from content** when access patterns and change rates differ: small, frequently-listed, slow-changing index vs. big, on-demand, fast-changing payload (separation of concerns).
5. **Cross-store pointers** are foreign keys generalized — a reference, not a copy; no cross-system integrity, so design for dangling pointers.
6. **A stateless LLM has no memory** — the transcript must be re-sent every call; decide *who holds it* (client array vs. managed service behind an id). The store is unbounded; the **context window** limits the model, not storage.
7. **Session / conversation / thread** are the same concept under different provider names — don't let the noun mislead you.
8. **Keep non-critical writes off the latency path** (background/green-thread the metadata write so it doesn't delay the user's first byte).
9. **SQL for relational + transactional + cross-entity queries; document store (Cosmos/DynamoDB/Mongo) for key-based access at scale and flexible shape.** Many systems use both.